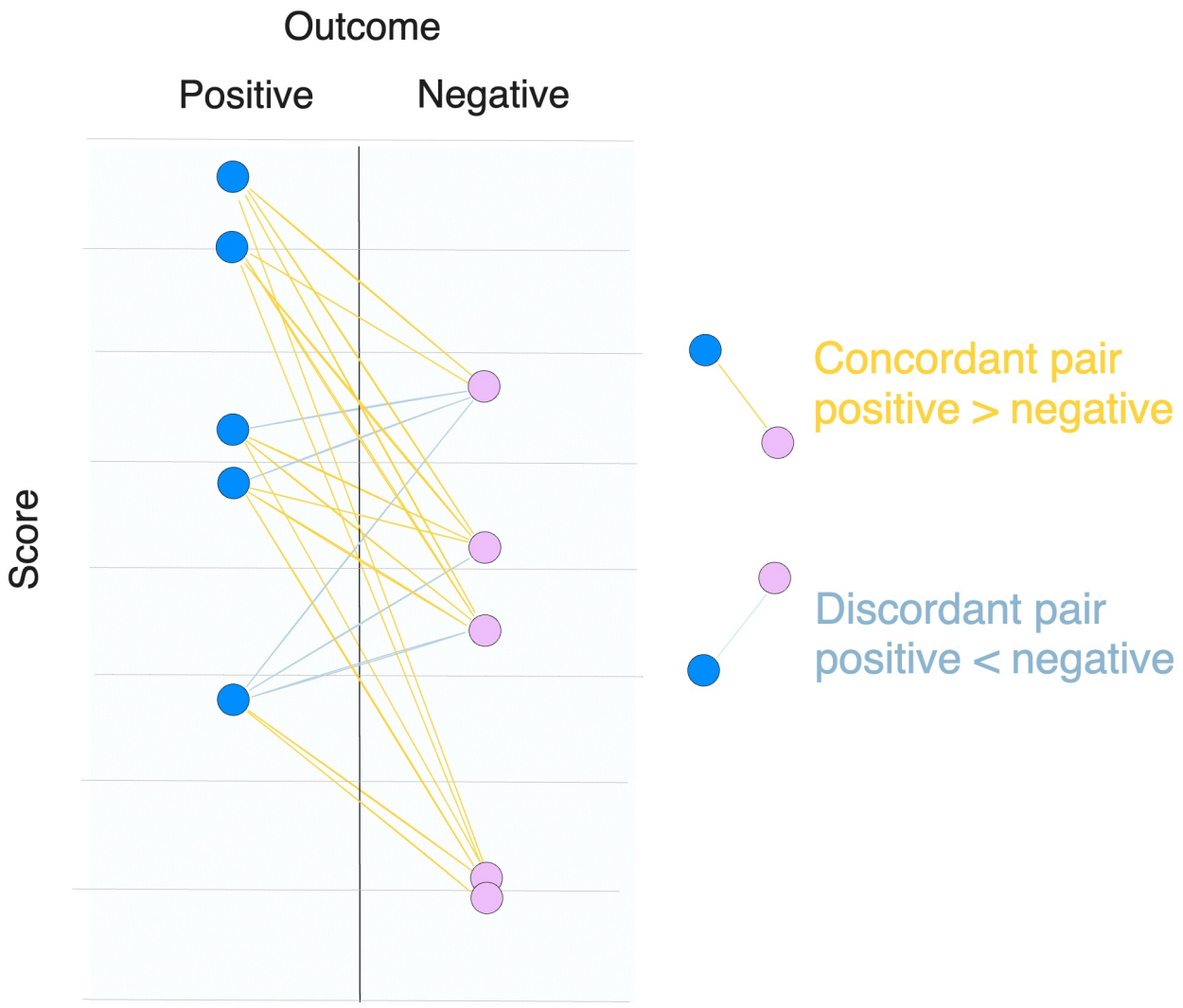
Somers' D


Outcome
Positive
Negative
Score
Concordant pair
positive > negative
Discordant pair
positive < negative

Somers' D in Risk Management Models

Somers' D is a metric used for assessing the discriminatory power of predictive risk models with both continuous and discrete outcomes (aka regression and classification)

Somers' D quantifies the ordinal association between the predictor X and target Y, for example, between estimated and ratings and defaults or loss-given-default (LGD) scores

For binary problems like credit scoring, Somers' D is better known as the Gini score or Accuracy Ratio (AR) measuring the rank-ordering power of a model

This makes the understanding of Somers' D highly valuable for those interested in the ranking ability of scoring models

(A) Binary outcome

Default	Rating 1	Rating 2	Rating 3
Y = 0	140	180	93
Y = 1	2	35	87

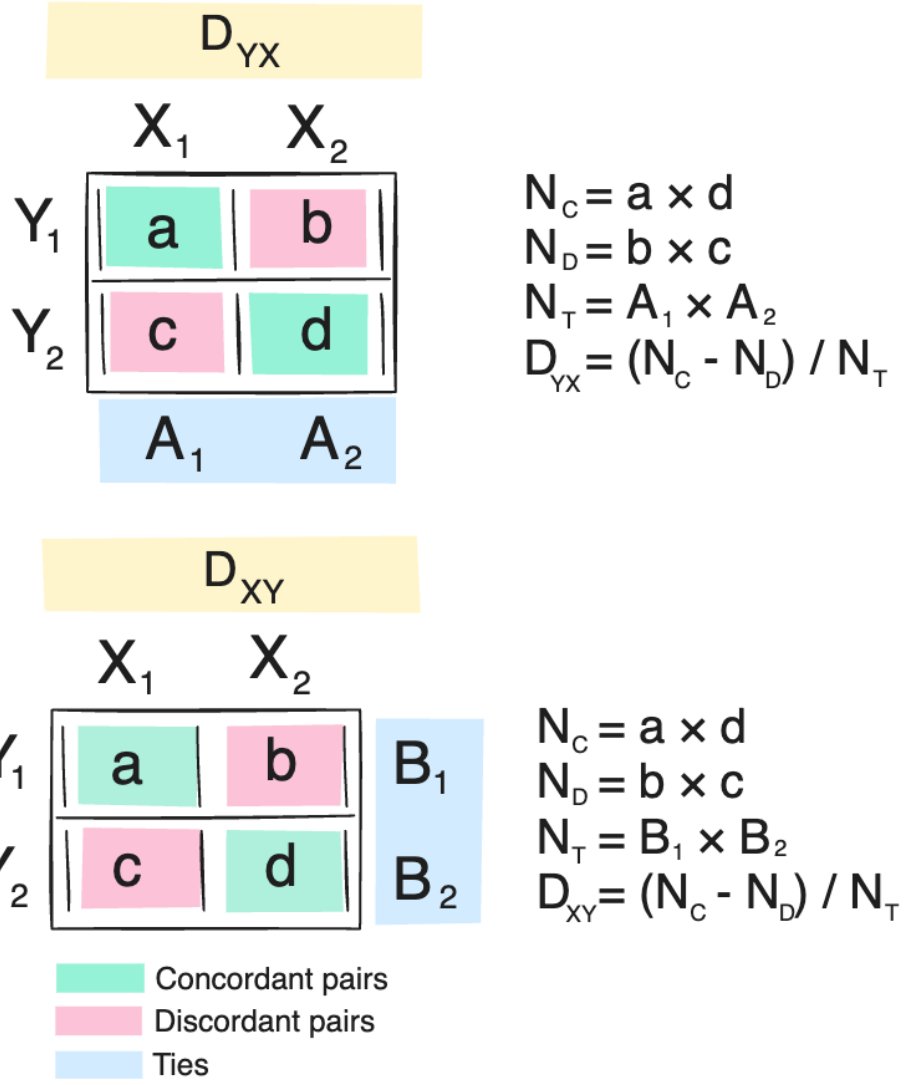
(B) Continuous outcome

Table 2.3: Example of a confusion matrix (count-based)

	Estimated LGD						
Realized LGD	LGD 1	LGD 2	LGD 3	LGD 4	LGD 5	LGD 6	Total
LGD 1	4	0	0	1	1	0	6
LGD 2	2	8	1	1	0	0	12
LGD 3	1	12	2	3	0	0	18
LGD 4	0	0	1	2	0	0	3
LGD 5	0	5	1	2	2	0	10
LGD 6	0	0	0	1	0	0	1
Total	7	25	5	10	3	0	50

1. Sproates C.L. Internal Risk Components Validation. https://essay.utwente.nl/71977/2/Sproates_MA_BMS.pdf

Two versions of Somers' D



The original paper by Robert Somers introduced the metric as a rank correlation coefficient that was able to measure ordinal association and not just independence with χ^2 test

There are two versions of Somers' D, $\mathbf{D_{YX}}$ and $\mathbf{D_{XY}}$, choice of which is determined by what we provide as X (predictor or the target variable), it only influences the denominator whether we count ties across rows or columns

The diagrams to the left show the examples provided by R. Somers how $\mathbf{D_{YX}}$ and $\mathbf{D_{XY}}$ are calculated

For the binary problem examples, we will use $\mathbf{D_{XY}}$ since it is the Gini score or Accuracy Ratio (AR) conventionally used in scoring

Two versions of Somers' D: example

Somers' $\mathbf{D_{XY}}$ (binary outcome example)

Default	p = 0.25	p = 0.5	p = 0.75
Y = 0	3	5	2
Y = 1	1	7	6

To calculate $\mathbf{D_{XY}}$ we need to find concordant pairs, discordant pairs, and ties on Y. The second example shows of $\mathbf{D_{YX}}$ when Y (LGD) is not binary. N_C and N_D are reduced for readability

D_{XY} for the scoring example:

$N_C = 3 \times 7 + 5 \times 6 + 3 \times 6 = 69$

$N_D = 5 \times 1 + 2 \times 7 + 2 \times 1 = 21$

$N_T = (3 + 5 + 2) \times (1 + 7 + 6) = 140$

$\mathbf{D_{XY} = (69 - 21) / 140 = 48 / 140 = 0.34}$

Somers' $\mathbf{D_{YX}}$ (continuous outcome example)

LGD rating	No collateral	Collateral
1	8	2
2	6	5
3	3	4
4	1	3
5	2	3
	20	17

D_{YX} for the LGD example[1]:

$N_C = 8 \times 5 + 8 \times 4 + 8 \times 3 + 8 \times 3 + 6 \times 4 \ldots = 201$

$N_D = 2 \times 6 + 2 \times 3 + 2 \times 1 + 2 \times 2 + 5 \times 3 \ldots = 72$

$N_T^2 = 20 \times 17 = 340$

$\mathbf{D_{YX} = (201 - 72) / 340 = 129 / 340 = 0.38}$

1. Example from the original Somers' paper, p. 806.
2. Tied on X, to calculate D_{XY} we would calculate sums across each row and multiply them with one another. See full calculation here.

Somers' D and AUC / Gini score

(A) Somers' D (Gini) in a contingency table

Default	Rating 1	Rating 2	Rating 3
Y = 0	140	180	93
Y = 1	2	35	87

```python
import numpy as np
from scipy.stats import somersd

# Create the data
data = [[140, 2],
        [180, 35],
        [93, 87]]

# Transpose the table
table = np.array(data).T
print(table)

# For Dxy, y is the row, x is the column
res = somersd(table)
print(f"{res.statistic:.4f}")
# [[140 180  93]
# [  2  35  87]]
# 0.5651
```

(B) Somers' D (Gini) on observation level

```python
import pandas as pd
from sklearn.metrics import roc_auc_score

# Define the distribution
distribution = {
    'Rating 1': [140, 2],
    'Rating 2': [180, 35],
    'Rating 3': [93, 87]
}

data = []
for rating, counts in distribution.items():
    rating_value = int(rating.split()[1])
    zeros = [0] * counts[0]
    ones = [1] * counts[1]
    data.extend([(rating_value, value) for value
in zeros + ones])

df = pd.DataFrame(data, columns=['Rating',
'Default'])

somers_d = roc_auc_score(df['Default'],
df['Rating']) * 2 - 1
print(f"{somers_d:.4f}")
# 0.5651
```

Implementation of Somers' D from scratch

```python
import itertools
import numpy as np

def somers_d_from_matrix(matrix: np.ndarray):

    # Calculate concordant and discordant pairs
    concordant_pairs = 0
    discordant_pairs = 0

    for i, j in itertools.combinations(range(len(matrix)), 2):
        concordant_pairs += np.sum(np.triu(
          np.outer(matrix[i], matrix[j]), 1)
          )
       discordant_pairs += np.sum(
          np.triu(np.outer(matrix[j], matrix[i]), 1)
          )

    # Calculate total pairs
    col_sums = np.sum(matrix, axis=0)
    total_pairs = np.sum(np.triu(np.outer(col_sums, col_sums), 1))

    somers_d = (concordant_pairs - discordant_pairs) / total_pairs

    return somers_d, concordant_pairs, discordant_pairs, total_pairs
```

This implementation gave me +30% faster speed than scipy's somersd ➡️

Model evaluation with Somers' D (LGD dataset[1])

Model evaluation

Model	Dxy
MLP	0.6133
HistGradientBoosting	0.6112
XGB Random Forest	0.6094
CatBoost	0.6055
Ridge	0.5978
Linear	0.5977
LightGBM	0.5955
XGBoost	0.5942
Random Forest	0.5796

Numpy implementation

```
CPU times: user 2min 30s, sys: 14.6 s, total: 2min 45s
Wall time: 2min 33s
```

Scipy implementation

```
CPU times: user 3min 7s, sys: 7.76 s, total: 3min 15s
Wall time: 3min 3s
```

1. Target: lgd_time. Bart Baesens et al. Credit Risk Analytics. LGD Dataset. http://www.creditriskanalytics.net/datasets.html

Optbinning example with LGD dataset[1]

```python
from optbinning import ContinuousOptimalBinning

variable_name = 'LTV'
x = X_train[variable_name].values
y = y_train.values

optb = ContinuousOptimalBinning(
    name=variable_name,
    dtype="numerical",
    min_bin_size=1e-1,
    max_n_bins=5
)

optb.fit(x, y)

# build a binning table
binning_table = optb.binning_table.build()
predicted_scores =
optb.transform(X_test[variable_name].values)
table = pd.crosstab(y_test,
predicted_scores).values

somers_d = somersd(table).statistic
somers_dxy = somers_d_from_matrix(table.T)[0]
print(f"Somers' D (scipy): {somers_d:.4f}")
print(f"Somers' D (numpy): {somers_dxy:.4f}")
```

lgd_pred	0.091234	0.130317	0.190938	0.420531	0.545768
lgd_time					
0.000010	41	21	71	10	6
0.000018	1	0	0	0	0
0.000135	0	0	1	0	0
0.000717	0	0	1	0	0
0.001137	1	0	0	0	0
...	...	...	...	...	...
0.994413	0	0	0	0	1
0.994834	0	0	0	1	0
0.995180	0	0	1	0	0
0.999873	0	0	0	1	0
0.999990	2	1	10	6	3

340 rows × 5 columns

Above we show how the contingency table between the LTV variable using 5 bins (columns in the contingency table) and LGD target variable

1. Target: lgd_time. Bart Baesens et al. Credit Risk Analytics. LGD Dataset. http://www.creditriskanalytics.net/datasets.html

Notebook

Notebook with the code available here ↘